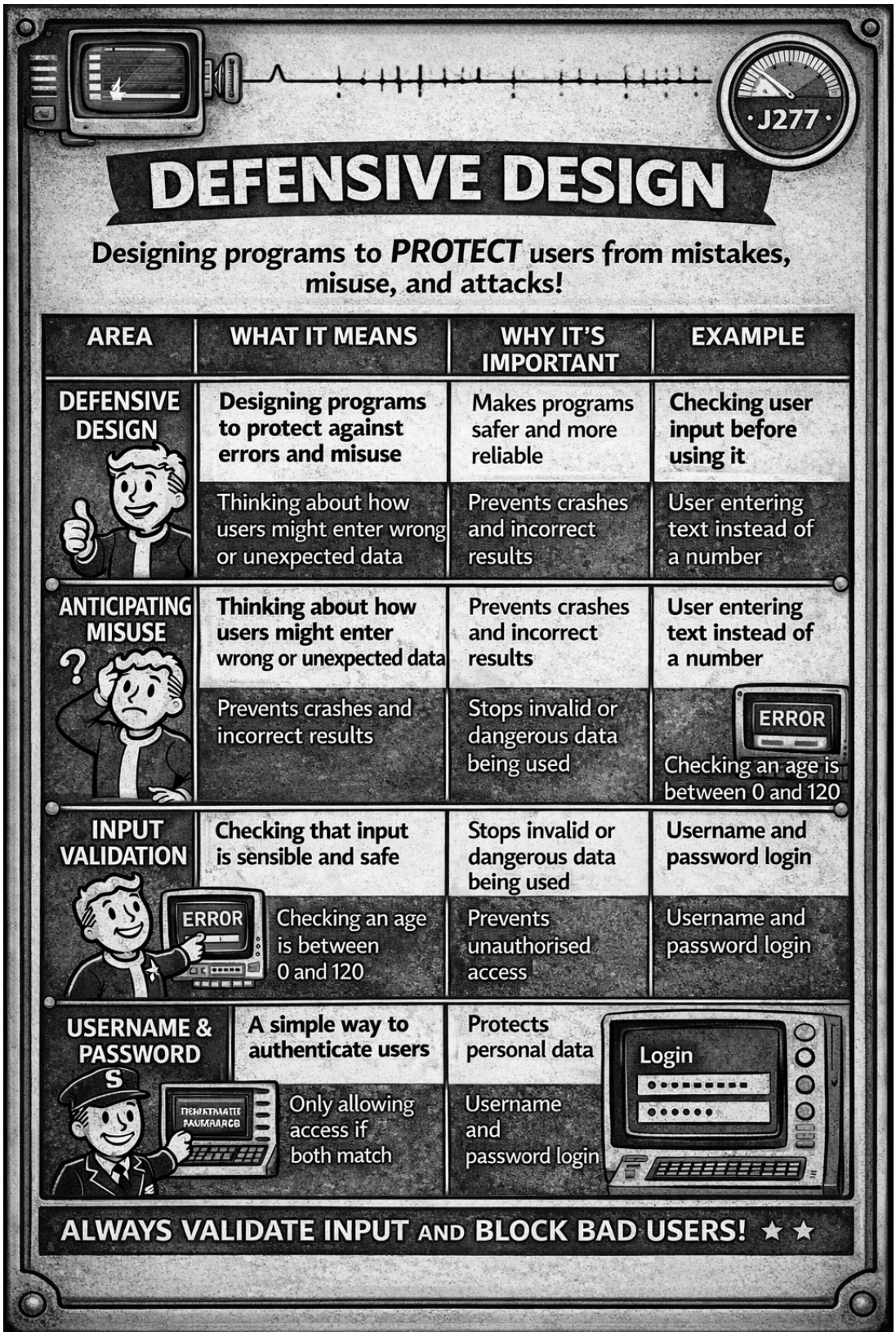
2.3 Producing Robust Programs

Keyword	Definition
Defensive Design	Designing and building computer code with the ability to keep it secure, easy to maintain and robust as a priority.
Authentication	A security mechanism which requires a password, code, fingerprint or other factor to gain access to all or part of a system.
Input Validation	The process of comparing data entered into the program to a set of rules to ensure it is valid and will not damage the system before processing it.
Maintainability	The ability to keep code updated, fix bugs and add new features easily.
Naming Conventions	An agreed way of naming parts of the program such as variables or sub programs such that they are easy to read and their purpose is clear.
Indentation	Inserting spaces at the start of a line or lines of code so to group them together and make the code easier to read.
Comment	Lines which do not form part of the code and are not executed but instead provide notes and explanations to those looking at the code.
Testing	The process of trialling a program in order to ensure it performs as expected.
Iterative Testing	Testing which is carried out whilst the program is being developed.
Final Testing	Testing which is carried out at the end of development just before the code is released.
Syntax Error	An error which breaks the rules of the programming language and stops the code from running.
Logic Error	An error which produces an unexpected output even though the code still runs.
Normal Data	Data which the program should accept without error.
Boundary Data	Data at the extremes of what the program should accept without error.
Invalid Data	Data of the correct type, but not valid for the program and which should be rejected.
Erroneous Data	Data which is the wrong type and should be rejected.

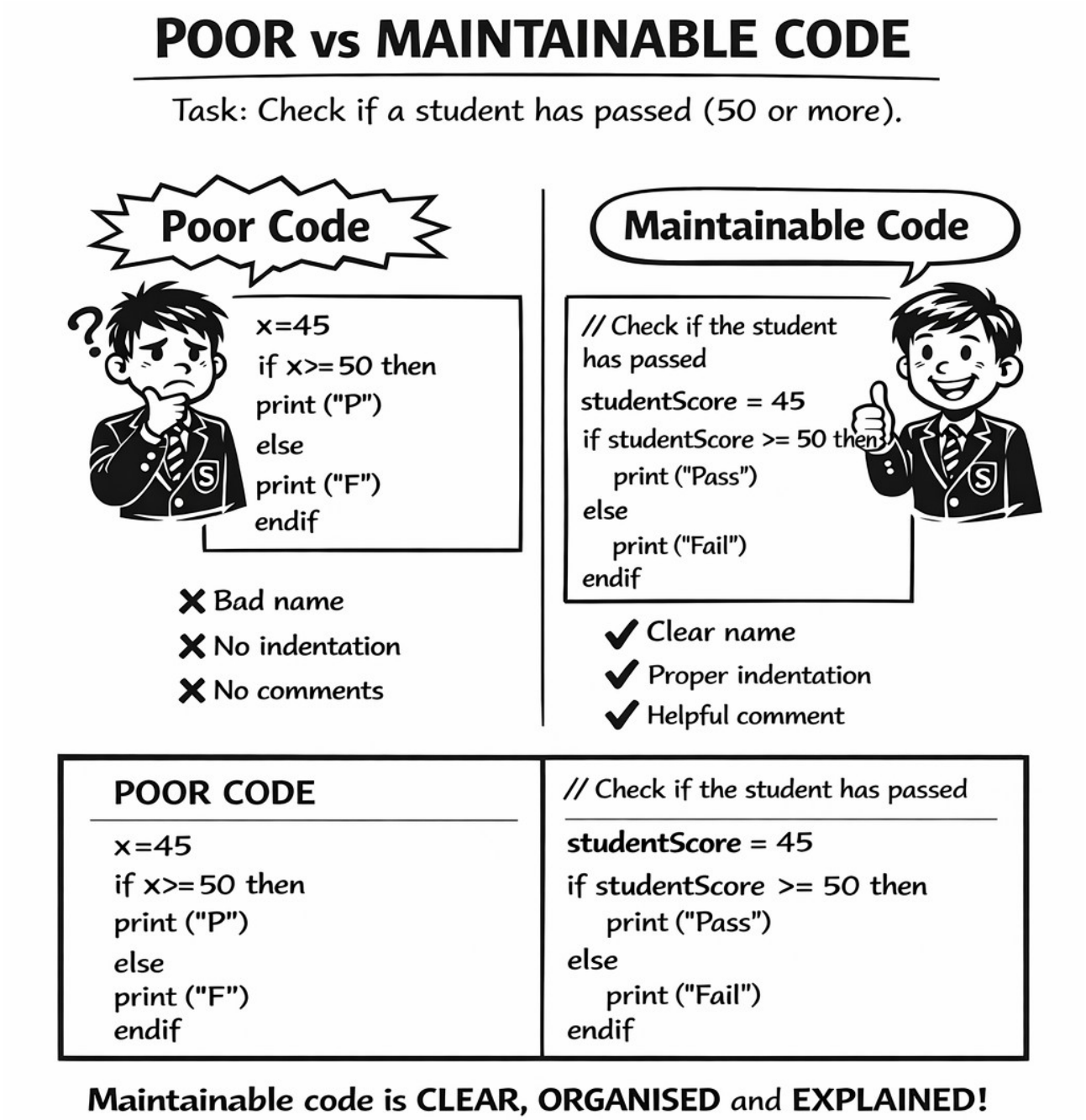
2.3.1 Defensive Design

2.3.1 Defensive design	
☐ Defensive design considerations: ○ Anticipating misuse ○ Authentication ☐ Input validation ☐ Maintainability: ○ Use of sub programs ○ Naming conventions ○ Indentation ○ Commenting	**Required** ✓ Understanding of the issues a programmer should consider to ensure that a program caters for all likely input values ✓ Understanding of how to deal with invalid data in a program ✓ Authentication to confirm the identity of a user ✓ Practical experience of designing input validation and simple authentication (e.g. username and password) ✓ Understand why commenting is useful and apply this appropriately



Basic	username = input("Enter username") password = input("Enter password") if username == "admin" AND password == "letmein" then print("Access granted") else print("Access denied") endif
Stops Brute force	correctUser = "admin" correctPass = "letmein" attempts = 0 maxAttempts = 3 loggedIn = FALSE while (attempts < maxAttempts) AND (loggedIn == FALSE) username = input("Enter username") password = input("Enter password") if (username == correctUser) AND (password == correctPass) then print("Access granted") loggedIn = TRUE else attempts = attempts + 1 endif endwhile if loggedIn == FALSE then print("Account locked") endif
Input validation (no blank entries)	username = input("Enter username") password = input("Enter password") if (username.length == 0) OR (password.length == 0) then print("Username and password cannot be blank") else if (username == "admin") AND (password == "letmein") then print("Access granted") else print("Access denied") endif endif
Multiple users (simple “database” using arrays)	array users[3] array passwords[3] users[0] = "admin" passwords[0] = "letmein" users[1] = "teacher" passwords[1] = "classroom" username = input("Enter username") password = input("Enter password") found = FALSE for i = 0 to users.length - 1 if (username == users[i]) AND (password == passwords[i]) then found = TRUE

	next i if found == TRUE then print("Access granted") else print("Access denied") endif



Feature	What it means	Why it is useful	Example
Subprograms	Breaking a program into smaller parts (procedures/functions)	Makes code easier to read, test, and reuse	`procedure calculateTotal(price, quantity)`
Naming conventions	Using clear, meaningful names for variables and subprograms	Helps others understand what the code does	`studentScore` instead of `x`
Indentation	Lining up code to show structure	Makes selection and loops easier to follow	Code inside `if/while/for` is indented

		logic and remind the programmer later	

2.3.1 Exam Questions on Defensive Design

Question	Tip to answer
1(a) What is input validation? **(1)** Input validation is checking data to make sure it meets the rules before a program accepts it.	"Check input meets rules."
1(b) Name one type of validation check. **(1)** Range check / Length check / Presence check.	Just name one.
1(c) What is authentication? **(1)** Authentication is checking a user's identity (e.g., username and password).	"Prove who you are."
1(d) Tick **one**: Which is usually more secure? **(1)** **A** Blacklist **B** Whitelist **B**	Whitelist = only allowed inputs.
2(a) State two ways a user could misuse an input box. **(2)** Enter letters when a number is needed.[1]Enter a very long string / symbols to try to break the program.[2]	Two short, different examples.
2(b) State two benefits of commenting code. **(2)** Makes code easier to understand for you/others.[1]Makes maintenance/debugging easier later.[2]	Think "understand" + "edit later".
2(c) A system accepts ages **0–120**. Give one **boundary** test value and one **invalid** test value. **(2)** Boundary: 0 (or 120).[1]Invalid: -1 (or 121).[2]	Boundary = edge. Invalid = outside.
3(a) Explain the difference between input sanitisation and input validation. **(3)** Sanitisation removes unwanted characters (cleans the input).[1]Validation checks the input matches rules (format/range/length).[2]Both help stop errors and reduce security risks.[3]	Sanitise = clean. Validate = check rules.
3(b) Explain why limiting login attempts improves security. **(3)** It reduces brute-force guessing of passwords.[1]After a few fails, the account can lock or delay attempts.[2]This makes it much harder for attackers to gain access.[3]	Mention brute force.
3(c) Explain how indentation and naming conventions improve maintainability. **(3)** Indentation shows which code is inside each IF/loop, so logic is clearer.[1]Good names (e.g., `totalTickets`) make the purpose obvious.[2]This helps others edit code safely and reduces mistakes.[3]	Readability + fewer bugs.
4(a) A program accepts whole numbers **1–100**. Write OCR Exam Reference Language to input `num`, and output **"accepted"** if valid, otherwise **"not accepted"**. **(4)** `num = int(input("Enter a number between 1 and 100"))`[1]`if num >= 1 AND num <= 100 then`[2] `print("accepted")`[3]`else`[4] `print("not accepted")`[3]`endif`	Two comparisons joined by AND. Include ELSE.
4(b) Write a **procedure** `showError()` that outputs **"INVALID INPUT"**, then call it. **(4)** `procedure showError()`[1] `print("INVALID INPUT")`[2]`endprocedure`[3]`showError()`[4]	Procedure = no return. Define then call.
4(c) Write a **function** `addVAT(price)` that returns `price * 1.2`. Then call it for `price =`	Function must RETURN. Use

Question	Tip to answer
`function addVAT(price)[1] withVAT = price * 1.2[2] return withVAT[3]endfunctiontotal = addVAT(50)[4]`	
6(a) A login system stores the correct password in `storedPass = "Synergy1"`. The user must enter a password. The program must: 1) remove spaces from the input, 2) allow **3 attempts**, 3) output **"Access granted"** if correct, otherwise **"Access denied"**, 4) after 3 failed attempts output **"Account locked"**. Write an algorithm (OCR Exam Reference Language). **(6)**	Count attempts. Loop. Only lock after 3 fails.
`storedPass = "Synergy1"[1]attempts = 0[2]access = False[3]while attempts < 3 AND access == False[4] userPass = input("Enter password")[5] userPass = userPass.replace(" ","")[6] if userPass == storedPass then[1] print("Access granted")[2] access = True[3] else[4] print("Access denied")[5] attempts = attempts + 1[6] endifendwhileif access == False then print("Account locked")endif`	
6(b) A ticket system starts with `tickets = 500`. A customer can book **1–10** tickets at a time. The program must: 1) ask for `num`, 2) validate `num` is 1–10, 3) if enough tickets remain, book them and subtract from `tickets`, otherwise output **"Not enough tickets"**, 4) repeat until `tickets` is 0. Write an algorithm (OCR Exam Reference Language). **(6)**	Validate 1–10 first, then check tickets >= num, loop until 0.
`tickets = 500[1]while tickets > 0[2] num = int(input("How many tickets (1 to 10)?"))[3] if num < 1 OR num > 10 then[4] print("Invalid amount")[5] else if tickets >= num then[6] print("Tickets booked")[1] tickets = tickets - num[2] else[3] print("Not enough tickets")[4] endif endifendwhile`	

02 KEY VOCABULARY / 2.3.1

Defensive design

Use the correct technical wording before attempting longer answers.

Validation

Checking input against rules such as range, length or type.

Authentication

Checking that a user really is who they claim.

Maintainability

Making code easier to read, understand and update.

Input sanitisation

Handling potentially dangerous input safely to reduce vulnerabilities.

Anticipating misuse

Planning for unexpected, incorrect or malicious actions.

03 WORKED EXAMPLE / 2.3.1

Defensive design

Follow a complete example. Notice the method before attempting the separate questions.

Task
Plan a secure registration form.

Think it through — small steps

1. Validate an age is within the permitted range.
2. Authenticate a returning user with an appropriate login method.
3. Avoid trusting inputs simply because they were submitted.
4. Use descriptive variable names and comments for non-obvious decisions.

Why this earns marks
The techniques protect against different risks: bad data, unauthorised access and hard-to-maintain code.

SPEC 2.3.1 | SKILL 22 OF 28

Defensive design

REMEMBER

Defensive design anticipates errors and misuse through input validation, authentication and maintainable code.

THINK OF IT LIKE...

Seatbelts for software: prevent foreseeable failures.

WORKED EXAMPLE / APPLY

Reject an age of -3 before storing it as a person's age.

YOU DO - EXAM PRACTICE

Distinguish validation from authentication. (2)

Write your answer before opening the separate answer section.

04 INDEPENDENT EXAM PRACTICE / 2.3.1

Defensive design

Do this without viewing the next answer pages. Use a whiteboard first for code/traces.

EXAM-STYLE CHALLENGE · 6 MARKS

A website registers students for a club. Describe three distinct defensive-design measures it should use, and explain how EACH would reduce risk or improve maintenance.

Your response

PREDICT → WRITE → TEST → IMPROVE | Self-mark after finishing the entire topic.

MARK SCHEME | SKILL 22 OF 28

Defensive design

QUESTION

Distinguish validation from authentication. (2)

MODEL ANSWER

Validation checks that data satisfies defined rules (1). Authentication verifies a user's claimed identity (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

05 ANSWERS & MARK SCHEME / 2.3.1

Defensive design

This answer belongs to the exam-style challenge on the previous practice page.

6-mark challenge · Model answer
Registration system safeguards

Mark 1 Measure: validate the age against an allowed range.

Mark 2 Reason: avoids invalid ages reaching the data store.

Mark 3 Measure: authenticate accounts appropriately before access.

Mark 4 Reason: reduces the risk of unauthorised access.

Mark 5 Measure: use meaningful names and clear structure (or comments).

Mark 6 Reason: helps future developers understand and safely update the code.

Accept equivalent correct methods, code or relevant wording. Award each point once.

2.3.2 Testing

2.3.2 Testing	
☐ The purpose of testing ☐ Types of testing: ○ Iterative ○ Final/terminal ☐ Identify syntax and logic errors ☐ Selecting and using suitable test data: ○ Normal ○ Boundary ○ Invalid/Erroneous ☐ Refining algorithms	**Required** ✓ The difference between testing modules of a program during development and testing the program at the end of production ✓ Syntax errors as errors which break the grammatical rules of the programming language and stop it from being run/translated ✓ Logic errors as errors which produce unexpected output ✓ Normal test data as data which should be accepted by a program without causing errors ✓ Boundary test data as data of the correct type which is on the very edge of being valid ✓ Invalid test data as data of the correct data type which should be rejected by a computer system ✓ Erroneous test data as data of the incorrect data type which should be rejected by a computer system ✓ Ability to identify suitable test data for a given scenario ✓ Ability to create/complete a test plan

TESTING (OCR J277)

What is testing?

Testing is checking that a program **works correctly, gives the right results,** and **does not crash.**

Purpose of testing

- Testing is used to:
  - Find errors and bugs
  - Check the program meets the requirements
  - Make sure the **program** works with **different inputs**
  - Improve **reliability** and **quality**

Testing helps prevent problems before users see them.

Testing during development vs final testing

Testing modules during development

This is testing **small parts** of the program as they are built.

- Done **while coding**
- Tests individual modules / subprograms
- Easier to **find where** errors occur
- Used to **fix problems early**

Example: Testing a login check before adding menus.

Testing at the end of production

This is testing **the whole program** together.

- Done after the program is finished
- Checks the program works as a complete system
- Uses realistic data

Example: Testing a full student system from login to saving results.

Types of testing

1. **Iterative testing** means testing little and **often** during **development.**
   - **Test** after **each change**
   - Errors are caught **early**

Most common type during coding.

Final (terminal) testing

2. **Tests the whole system**
   - Uses **normal, extreme,** and **invalid data**
   - Confirms the program is ready for use

Happens at the ***end*** of development.

Programs can contain **errors** that stop them working properly.

The two main types you need to know are **syntax errors** and **logic errors**. A **syntax error** is a mistake that **breaks the rules of the programming language** A **logic error** is when the program **runs**, but gives the **wrong or unexpected output**.

Feature	Syntax error	Logic error
What it means	Breaks the grammatical rules of the programming language	The program follows the rules but the logic is wrong
Does the program run?	❌ No	✔️ Yes

OCR-style example	`if score >= 50`
	`  print("Pass")`
	`endif`
		`if score >= 50 then`
		`  print("Pass")`
		`else`
		`  print("Pass")`
		`endif`

What happens	Program will not run because `then` is missing	Program runs but always prints `"Pass"`
Common mistakes	Missing `then`, `endif`, brackets, or quotation marks	Using the wrong condition or same output in both choices

Feature	Syntax error	Logic error
What it means	Breaks the grammatical rules of the programming language	The program follows the rules but the logic is wrong
Does the program run?	❌ No	✔️ Yes
OCR-style example (FOR loop)	for i = 1 to 5 print(i)	for i = 1 to 5 print(i)next i
What happens	Program will not run because `next i` is missing	Program runs but produces the wrong result
Common mistakes	Missing `next i`; incorrect FOR syntax	Using wrong start/end values; incorrect step value

Feature	Syntax error	Logic error
What it means	Breaks the grammatical rules of the programming language	The program follows the rules but the logic is wrong
Does the program run?	❌ No	✔️ Yes
OCR-style example (WHILE loop)	while count < 5 print(count) endwhile	count = 1 while count <= 5 print(count) endwhile
What happens	Program will not run because `count` was never defined	Program runs forever because `count` never changes
Common mistakes	Missing `count = 1` before the loop; missing `endwhile`	Forgetting `count = count + 1`; using a condition that never becomes FALSE

Testing – practise different boundaries: An integer input accepts values from 12 to 60 inclusive. Classify 35, 12, 60, 61 and "twelve" as normal, boundary or invalid/erroneous where appropriate. Explain why the exact minimum and maximum are valid boundary tests, but values just outside the range must be rejected.

SELECTING AND USING TEST DATA (OCR J277)

What is test data?

Test data is input used to check that a **program works correctly** and safely.

Good testing uses **different types** of data to check all situations.

1 Types of test data

Normal test data:

- is **valid**
- is **expected**
- Should be **accepted** by the program
- Should not cause errors



Used to check the program works in normal conditions.

1 Normal test data

- Breaks (f **valid**
- The smallest or largest **valid value**
- Right on the **edge** of what is **allowed**
- Still **valid** and should be accepted

Used to check the program works in normal conditions.

2 Boundary test data

- **The smallest or largest valid value**
- Right on the **edge** of what is allowed
- Still **valid** and **should be accepted**

Used to check the program handles limits correctly.

3 Invalid test data

- The **correct data type.**
- But outside the allowed range
- Should be **rejected** by the program

Used to check **validation** rules.

Choosing suitable test data

1. **Identify the valid range.**
2. Pick at least one **normal value.**
3. **Pick both boundary values.**
4. Pick **one invalid** and one **erroneous** value.

Test data is input used to check that a program works correctly.

Remember BIN...

- **B** **Boundary** – edges
- **1** **Invalid** – wrong value
- **E** **Erroneous** – wrong type

One-sentence exam question

Test data is input used to check that a program works correctly

2.3.2 Exam Questions on Testing

Question	Tip to answer
1(a) What is testing? **(1)**	"Check it works."
Testing is checking a program works correctly and finds errors before it is used.	
1(b) Name one type of testing. **(1)**	Just name one.
Iterative / Final (alpha) / Final (beta).	
1(c) What is a syntax error? **(1)**	"Breaks the rules of the language."
A syntax error is when code breaks the rules of the programming language (e.g. missing a bracket).	
2(a) State two reasons why programs are tested. **(2)**	Two clear reasons.
To find and fix errors/bugs.[1]To make sure the program meets the requirements / works as expected.[2]	
2(b) A program accepts whole numbers **1–100**. Give one **normal**, one **boundary**, and one **invalid/erroneous** test value. **(3)**	Normal = typical. Boundary = edge. Invalid = outside.
Normal: 27.[1]Boundary: 1 (or 100).[2]Invalid/erroneous: 0 (or 101 / "Hello").[3]	
2(c) State two differences between alpha testing and beta testing. **(2)**	Alpha = in-house. Beta = real users.
Alpha is done by developers/testers before release.[1]Beta is done by real users in real situations after alpha.[2]	
3(a) Explain why boundary test data is useful. **(3)**	Edges often break code.
Boundary data checks the extreme valid values.[1]It finds off-by-one mistakes (e.g. using < instead of <=).[2]This helps ensure validation rules are correct.[3]	
3(b) Explain the difference between a logic error and a runtime error. **(3)**	Logic = wrong result. Runtime = crashes while running.
A logic error means the program runs but gives the wrong output.[1]A runtime error happens while the program is running and may crash (e.g. divide by zero).[2]Logic errors are found by testing outputs; runtime errors are triggered by certain inputs/conditions.[3]	
4(a) A program asks for an age (0–120). Write OCR Exam Reference Language to: input age, validate the range, and output **"accepted"** or **"not accepted"**. **(4)**	Two comparisons joined by AND.
`age = int(input("Enter age (0 to 120)"))`[1] `if age >= 0 AND age <= 120 then`[2] `  print("accepted")`[3] `else`[4] `print("not accepted")`[3] `endif`	
4(b) A programmer fixes a bug, then tests again straight away. Name the testing approach and explain why it is used. **(4)**	Name + why (repeat improve).
Iterative testing.[1]After each change, you re-test to check the fix works.[2]It also checks you haven't created new bugs elsewhere.[3]This improves reliability before final release.[4]	
6(a) A program should accept a username that is **6–12 characters** long and not blank. Write an algorithm (OCR Exam Reference Language) that: inputs `username`, uses a loop to keep asking until it is valid, then outputs **"Username accepted"**. **(6)**	Use WHILE with OR conditions.

Question	Tip to answer

```
username = input("Enter username")[1]
while username.length < 6 OR username.length > 12[2]
  print("Invalid username")[3]
  username = input("Enter username")[4]
endwhile[5]
print("Username accepted")[6]
```

6(b) A program asks for a number **1–100**. Create a test plan table (in words) that includes **at least 2 normal**, **2 boundary**, and **2 invalid/erroneous** test values, and what the expected outcome is for each. **(6)**

Include expected result every time.

Normal: 27 → accepted.[1]
Normal: 85 → accepted.[2]
Boundary: 1 → accepted.[3]
Boundary: 100 → accepted.[4]Invalid/erroneous: 0 → not accepted.[5]
Invalid/erroneous: 101 (or “Hello”) → not accepted.[6]

02 KEY VOCABULARY / 2.3.2

Testing and debugging

Use the correct technical wording before attempting longer answers.

Normal data

Typical valid input within the accepted range.

Boundary data

Values exactly at and near the edges of an accepted range.

Invalid data

Data outside the specified permitted values.

Erroneous data

Input of an unsuitable type or format.

Syntax error

A violation of a language rule that prevents correct parsing.

Logic error

Program runs but produces a result that does not meet the requirements.

03 WORKED EXAMPLE / 2.3.2

Testing and debugging

Follow a complete example. Notice the method before attempting the separate questions.

Task
Test a permitted age of 10 through 20 inclusive.

Think it through — small steps

1. Try normal data, e.g. age 15.
2. Try the two accepted boundaries, ages 10 and 20.
3. Try outside the boundaries, ages 9 and 21.
4. Try erroneous data such as the text "cat".
5. Check that a condition uses >= 10 rather than > 10 if age 10 is permitted.

Why this earns marks
The set tests a normal case, the boundaries, invalid values and an unsuitable data type.

SPEC 2.3.2 | SKILL 23 OF 28

Testing and test data

REMEMBER

Normal, boundary, invalid and erroneous test data reveal different faults.

THINK OF IT LIKE...

Test a bridge with everyday traffic and edge loads.

WORKED EXAMPLE / APPLY

For valid ages 13..18: test 15, 13, 18 and 19.

YOU DO - EXAM PRACTICE

For accepted ages 13 to 18 inclusive, give one normal input and one invalid input. (2)

Write your answer before opening the separate answer section.

SPEC 2.3.2 | SKILL 24 OF 28

Syntax and logic errors

REMEMBER

Syntax errors break language rules. Logic errors allow a program to run but produce the wrong result.

THINK OF IT LIKE...

A typo in the recipe versus the wrong ingredient amount.

WORKED EXAMPLE / APPLY

IF score > 50 may wrongly reject a score of exactly 50 if 50 should pass.

YOU DO - EXAM PRACTICE

Explain why using > instead of >= in a pass-mark check can be a logic error. (2)

Write your answer before opening the separate answer section.

04 INDEPENDENT EXAM PRACTICE / 2.3.2

Testing and debugging

Do this without viewing the next answer pages. Use a whiteboard first for code/traces.

EXAM-STYLE CHALLENGE · 6 MARKS

A booking form accepts whole-number ages 13 to 18 inclusive. Give one normal value, the two valid boundary values, two invalid values and one erroneous value. Label each.

Your response

PREDICT → WRITE → TEST → IMPROVE | Self-mark after finishing the entire topic.

MARK SCHEME | SKILL 23 OF 28

Testing and test data

QUESTION

For accepted ages 13 to 18 inclusive, give one normal input and one invalid input. (2)

MODEL ANSWER

Normal: 15 (or 14, 16 or 17) (1). Invalid: 12 or 19 (or another outside-range value) (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

MARK SCHEME | SKILL 24 OF 28

Syntax and logic errors

QUESTION

Explain why using > instead of >= in a pass-mark check can be a logic error. (2)

MODEL ANSWER

The program may still run without a syntax error (1). A mark equal to the threshold would be incorrectly rejected (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

05 ANSWERS & MARK SCHEME / 2.3.2

Testing and debugging

This answer belongs to the exam-style challenge on the previous practice page.

6-mark challenge · Model answer
Create a range-test plan

Mark 1 Normal: any whole number strictly between 13 and 18, e.g. 15.

Mark 2 Lower boundary: 13.

Mark 3 Upper boundary: 18.

Mark 4 Invalid below range: 12 or less.

Mark 5 Invalid above range: 19 or more.

Mark 6 Erroneous: non-numeric input such as "blue".

Accept equivalent correct methods, code or relevant wording. Award each point once.